Recovery capability: from "we have backups" to "we can restore"

> **Belona Cybersecurity** · July–October 2026 Real case: production platform of a small law firm (Client A). *Versión en español:* `../recuperacion-ante-desastres.md`

The problem

A small law firm runs a system holding **client case files as PDFs** and their associated data. There were no backups. If someone deleted something by mistake, or a process corrupted it, there was no way back.

For that firm this isn't an IT risk — it's a **professional one**. Losing a client's case file isn't fixed with an apology.

But the real problem was subtler than "backups are missing," and it shows up in almost every company that does have them:

> **Having copies and being able to recover are not the same thing**, and almost nobody verifies the second.

The reasoning, before the tooling

These decisions are identical on Google Cloud, AWS, Azure or a self-hosted server. **The platform only changes the commands.**

The two questions that decide everything

	What it asks	What it drives
RPO	how much data can I afford to lose?	how often you copy
RTO	how long can I be down?	how you restore

A practice receiving a handful of documents a day tolerates an RPO measured in hours. One billing by the minute does not. **Nobody can answer those two questions on the client's behalf** — they're business decisions dressed as technical ones, and asking them is half the work.

The 3-2-1 rule

> **3** copies · on **2** different media · with **1** off-site

The part that always gets skipped is the last one. A copy in the same account, region or provider as the original **shares its fate**: if account access is lost — a billing error, a compromised credential, a suspension — both go at once.

What was implemented

Layer	What it protects	Window
Point-in-time recovery	the whole database	**7 days**
Scheduled daily backups	the database	**14 days**
File versioning	the PDF case files	**90 days**
Off-site copy	everything, in a separate storage account	weekly
Source code	the application	full history in Git

RPO ~0 within the 7-day window · **RTO ~minutes** given the database size.

The layers aren't redundant: each covers a failure the others don't.

- **Point-in-time** helps when you know *when* things broke — "at 10:15 it was fine."
- **Daily backups** help when you find out late.
- **Versioning** recovers a single file without restoring anything else.
- **The off-site copy** helps when the problem is the whole account.

Restoring an entire database to recover one PDF is demolishing the house to change a lightbulb. That's why there are four layers, not one.

The finding: what was documented wasn't what was real

During the October verification the three status checks were run. Two confirmed expectations. **The third did not.**

```
Point-in-time recovery:  ENABLED          ✅
Daily backups:           14-day retention ✅
File versioning:         (not configured) ❌
```

File versioning was never enabled. The documentation listed it as implemented, and the "recover a deleted document" procedure depended on it — so that procedure **would not have worked.**

Three months of confidence resting on a control that didn't exist.

The nuance, which matters

The first reading was *"a deleted file is gone."* **That was overstated.**

Requesting the storage configuration in full — instead of filtering it — surfaced a **7-day soft-delete policy** the provider enables by default. There was a safety net nobody had documented.

The finding's real size:

	Before	After
Recovery window for a case file	**7 days**	**90 days**

It remains a finding — the documentation was wrong and the procedure was broken — but it wasn't total loss. **Stating it precisely matters more than stating it forcefully.**

Remediated

Versioning was enabled and a lifecycle rule added to delete non-current versions after 90 days. Without that rule, every edit of a document would keep a copy **forever** and storage would grow without bound.

```
versioning_enabled: true
lifecycle: delete non-current versions > 90 days
```

⚠️ **Versioning only protects from the moment it's enabled.** Anything deleted before that doesn't come back.

How it was found — the reusable part

The finding didn't come from an audit or a scanner. It came from **going to run the restore test** and starting by verifying that what was documented actually existed.

Two details of method proved decisive:

1. Request the full output, not a filtered one. The first query used a field-name filter and returned `null`. That was read as *"it's disabled."* The real field had a different name: `null` meant *"that field doesn't exist,"* not *"the control is off."*

The unfiltered output resolved the ambiguity **and surfaced the soft-delete policy** nobody had mentioned — the fact that corrected the severity.

> A filter hides exactly what you didn't think to ask for.

2. Don't trust a negative result without testing the tool. Earlier in this same project, documents were checked for certain text using a tool that **can't read that format**. It found nothing because it couldn't look, not because there was nothing.

> A "found nothing" is only worth something if the tool finds something it **should** find.

What's still pending

The full restore test has not been executed. The backups are configured and verified to exist; that they can be restored end to end is **inferred, not measured**.

It's the main outstanding item of this work and it's stated here deliberately:

> **A backup that has never been restored isn't a verified backup. It's an assumed one.**

The procedure is written — restore to a temporary database, validate, delete the temporary one — and it never touches production. Pending execution and recording.

Two minor observations from the same review:

- Storage uses per-object access control lists rather than unified permissions. Two permission systems coexisting is easier to get wrong than one.
- The primary bucket is single-region. Worth confirming the backup bucket really is multi-region as documented — **after today, that claim doesn't get taken on faith.**

What a client receives

1. The two questions answered **by them**: how much data they can lose and how long they can be down.
2. The implemented layers, **verified against the system's own output**, not against a settings screen.
3. **The restore test run in front of them**, or the date it will be.
4. The written procedure, so someone who wasn't there can execute it.

Point 3 is the one almost nobody delivers. **It's also the only one that proves the rest works.**

Appendix — the Google Cloud implementation

The concrete commands for this case. **The equivalent exists on AWS** (S3 versioning, AWS Backup, lifecycle policies) **and on Azure** (blob versioning, Azure Backup). What doesn't change between platforms is everything above.

```
# Verify each layer actually exists
gcloud firestore databases describe --database="(default)" \
  --format="value(pointInTimeRecoveryEnablement)"
```

```
gcloud firestore backups schedules list --database="(default)"
gcloud storage buckets describe gs://<BUCKET> --format=json   # full, unfiltered

# Enable versioning + retention for old versions
gcloud storage buckets update gs://<BUCKET> --versioning
gcloud storage buckets update gs://<BUCKET> --lifecycle-file=lifecycle.json

# Restore to a NEW database. Never over the original.
gcloud firestore backups list --format="table(name,snapshotTime,state)"
gcloud firestore databases restore \
  --source-backup="<BACKUP>" --destination-database="restore-tmp"

# Recover a single file from a previous version
gcloud storage ls -a "gs://<BUCKET>/<PATH>"          # includes deleted versions
gcloud storage cp "gs://<BUCKET>/<PATH>#<GENERATION>" "gs://<BUCKET>/<PATH>"
```

The platform-independent rule: restore to a new destination and validate before touching the original. Importing over production "to see if it works" is how a recovery drill becomes the incident.

From the technical portfolio of Belona Cybersecurity. Real work on a production platform, performed under authorization from the owner.
